Multivariate Kalman Filter

Euan Riach

2026

1 Introduction

The Kalman Filter is an algorithm for filtering noise. This implementation is multivariate and linear. For non-linear systems an extended Kalman filter (EKF) is required. The EKF is not covered here.

2 Algorithm

This algorithm mostly comes from [2]. The Kalman filter repeats a two step process to estimate the state $\boldsymbol{x}$, an $n \times 1$ vector, of a system. First, prediction produces an a priori estimate using the filter's previous estimate. The filter then updates the a priori estimate by combining it with a measured value. The measurement does not need to cover all the state space.

2.1 Prediction

For notation, we will use $\hat{\boldsymbol{x}}_k^-$ to represent the a priori estimate of $\boldsymbol{x}$ at time k and $\hat{\boldsymbol{x}}_k$ for the a posteriori estimate. The filter requires a confidence in its current estimate, this is represented by the $n \times n$ error covariance matrix P_k, similarly P_k^- will represent the a priori estimate of P_k and P_k the a posteriori. The equations for the prediction step are

$$\hat{\boldsymbol{x}}_k^- = F\hat{\boldsymbol{x}}_{k-1} + B\boldsymbol{u}_k \tag{1}$$

$$P_k^- = FP_{k-1}F^T + Q. \tag{2}$$

The matrix $n \times n$ F is the state transition matrix, it is the linear equations used to model the system. $\boldsymbol{u}_k$ is a vector $l \times 1$ representing some driving force in time k and the matrix $n \times l$ B converts the measurement into the state space. Q is an $n \times n$ covariance matrix of the process noise.

2.2 Update

The filter then acquires an $m \times 1$ vector of some observation of the system which is subject to measurement variance. We first require the $n \times m$ Kalman gain

matrix, K. The Kalman Gain tells the filter how much weighting to give to the a priori estimate and observation. It is given by

$$K_k = P_k^- H^T (H P_k^- H^T + R)^{-1} \tag{3}$$

where H is the $m \times n$ matrix which transfers the observation to the state space and R is the $m \times m$ measurement noise covariance matrix. The filter can then use the Kalman gain to update the a priori estimates to give us a posteriori estimates:

$$\hat{\boldsymbol{x}}_k = \hat{\boldsymbol{x}}_k^- + K_k(z_k - H\hat{\boldsymbol{x}}_k^-) \tag{4}$$

$$P_k = (I - K_k H)P^-(I - KH)^T + KRK^T \tag{5}$$

Note: there are two formulas for P_k, we will use the Joseph form as it is more robust [1].
See appendix A for python implementation.

3 Demo - Tracking a Ball

3.1 Setup

To demonstrate the filter I will use the case of a ball moving in the xy plane confined to a box. Our state space will be

$$\boldsymbol{x} = \begin{pmatrix} x \\ y \\ v_x \\ v_y \end{pmatrix}.$$

Let the motion be governed by the linear equations

$$x_k = x_{k-1} + v_x h$$
$$y_k = y_{k-1} + v_y h$$

where v_x and v_y represent the velocity in the x and y directions and are assumed to be constant and h is some time step. Hence, we can define the state transition matrix as

$$F = \begin{pmatrix} 1 & 0 & h & 0 \\ 0 & 1 & 0 & h \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

The ball is going to be confined in a box. If it hits the boundary it will bounce off the edge without losing energy.

When the ball is moving there will be added noise to the velocities to simulate small deviations due to external forces. Our observations will only be of the position so

$$z = \begin{pmatrix} x \\ y \end{pmatrix} + \text{noise}$$

and our measurement function will need to convert it to the state space, thus,

$$H = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{pmatrix}.$$

When implementing the filter we must define the process covariance matrix appropriately. I have assumed there is no correlation between x and y measurements but there is correlation between position and velocity in the same axis. I have also added more variance in velocity to account for the ball bouncing at the boundary. Hence,

$$Q = \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 2 & 0 \\ 0 & 1 & 0 & 2 \end{pmatrix} * q$$

where q is some scalar. The code for this is in appendix B.

3.2 Results and Improvements

The filter works as intended. As can be seen from figure 1, the filter estimate accurately tracks the true position and ignores sensor noise. The limitations occur at the boundaries as there is a lag whenever the ball changes direction, this can be seen in the spikes in the residuals in figure 2. From figure 3, we see the Kalman Gain reaches a steady state after 250 iterations. The filter reaches a set confidence in its estimate. This is not ideal for this system. An improvement would be to have the filter add additional process noise whenever it estimates it is close to the boundaries, this would cause the Kalman gain to increase and give the filter more faith in the measurements when it is near a boundary.

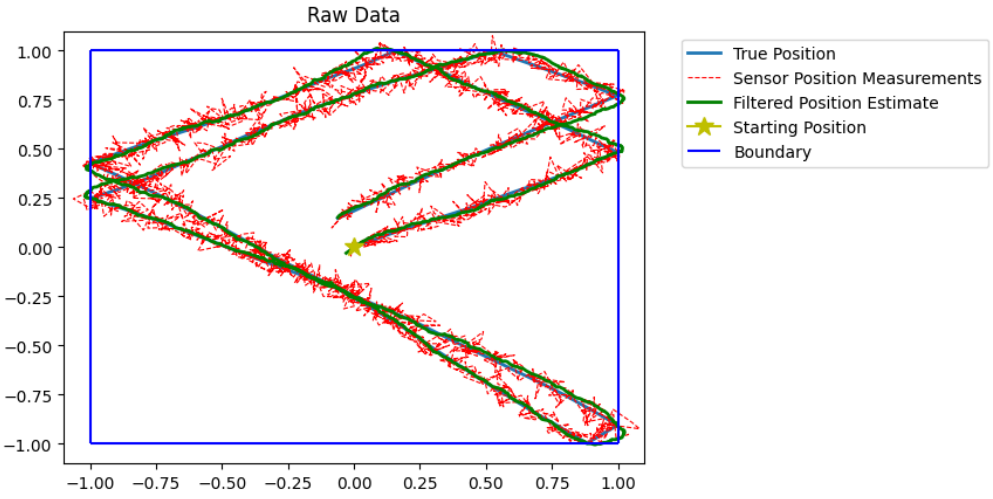


Figure 1: Path Taken

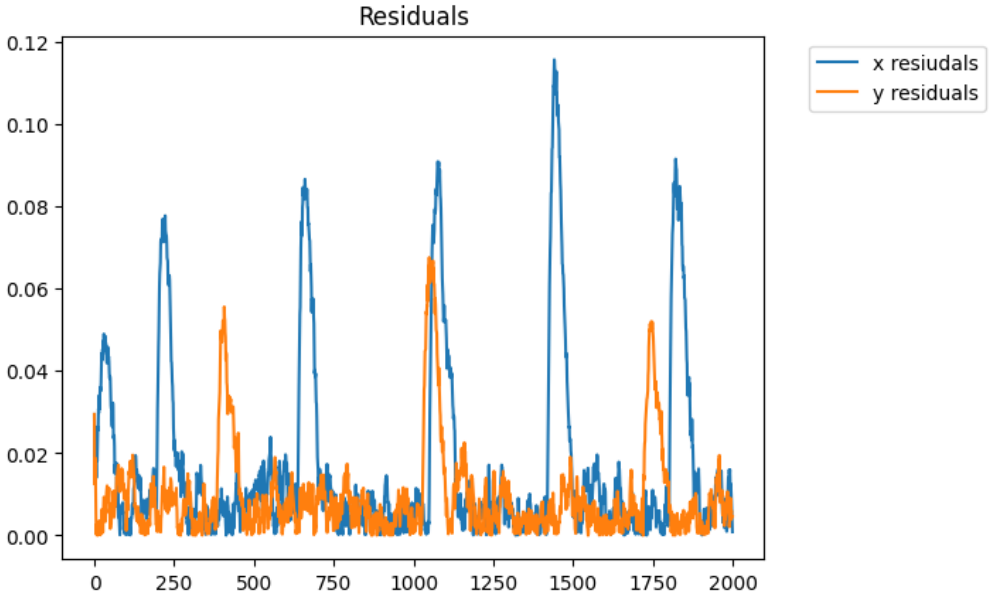


Figure 2: Residuals

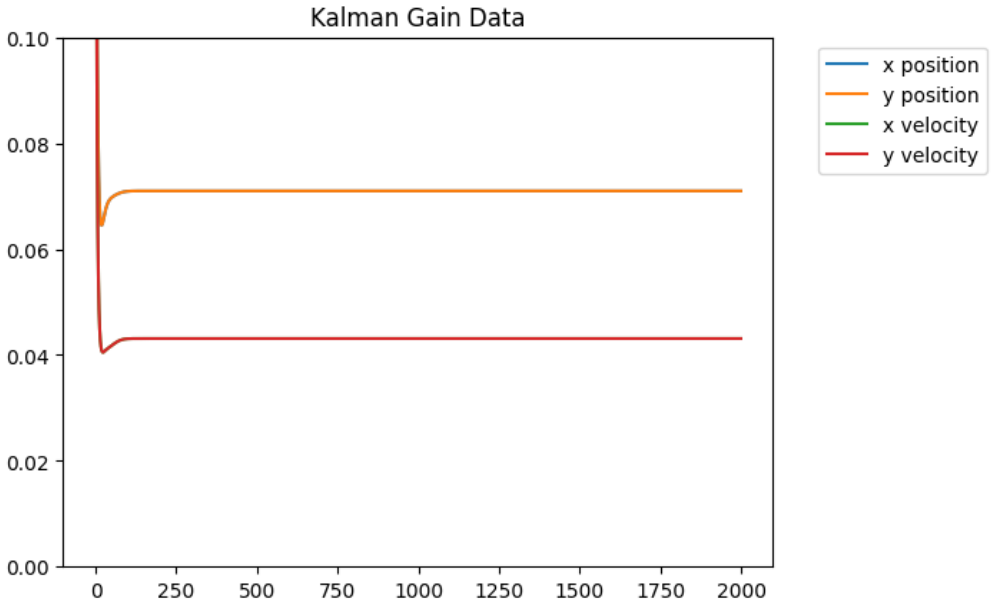


Figure 3: Kalman Gain for the State Space

4 Next Steps

Following on from this, building an extended Kalman filter would allow the modelling of more realistic systems. An EKF follows the same principle as the standard Kalman filter but requires linearising the function at each step using the Jacobian in place of the state transition matrix. Other applications of the multivariate Kalman filter include estimating volatility in equities by filtering out transaction noise and estimating the hedge ratio, β, of correlated assets.

References

[1] Roger Labbe. Kalman and bayesian filters in python, 2015.

[2] Greg Welch and Gary Bishop. An introduction to the kalman filter greg welch 1 and gary bishop 2 tr 95-041, 2001.

Appendix A - Multivariate Kalman Filter Class

```
import numpy as np

class KalmanFilter:
    def __init__(self, dim_x, dim_z, dim_u = 0):
        """
        The following are place holder objects and must be defined appropriately after
        calling the class.
        dim_x is the dimension of the state vector
        dim_z is the dimension of the measurement
        dim_u is the dimension of the driving force
        """
        self.x = np.zeros((dim_x, 1))       #State
        self.P = np.eye(dim_x)              #Error Covariance Matrix
        self.Q = np.eye(dim_x)              #Process Variance
        self.R = np.eye(dim_z)              #Measurement Variance
        self.u = np.zeros((dim_u, 1))       #Driving Force
        self.B = np.zeros((dim_x, dim_u))   #Control Transition Matrix
        self.F = np.zeros((dim_x, dim_x))   #State Transition Matrix
        self.H = np.zeros((dim_z, dim_x))   #Measurement Function
        self._I = np.eye(dim_x)             #Identity Matrix
        self.K = np.zeros((dim_x, dim_z))   #Kalman Gain
        self.dim_z = dim_z

    def predict(self):
        """
        Filter produces an a priori estimate of the next position off of the current
        estimate.
        """
        self.x = (self.F @ self.x) + (self.B @ self.u)
        self.P = (self.F @ self.P @ self.F.T) + self.Q

    def update(self, z):
        """
        Filter produces an a posteriori estimate by combining the a priori estimate
        with an observation.
        z - observation
        """
        z = np.asarray(z).reshape(self.dim_z, 1)
        self.K = self.P @ self.H.T @ np.linalg.inv(self.H @ self.P @ self.H.T + self.R)
        self.x = self.x + self.K @ (z - self.H @ self.x)
        self.P = (self._I - self.K @ self.H) @ self.P @ (self._I - self.K @ self.H).T
                + (self.K @ self.R @ self.K.T)
```

Appendix B - Ball Tracking

```
import numpy as np
import matplotlib.pyplot as plt

#Sensor object taking in a true position and adding white noise.
class Sensor:
    def __init__(self, measure_variance):
        self.r = measure_variance

    def sense(self, x):
        return x + np.random.normal(0, np.sqrt(self.r), size = x.shape)

#Ball object simulating motion of a ball confined to a box, assuming no energy loss.
class Ball:
    def __init__(self, initial_velocity, initial_pos, x_bounds, y_bounds, F,
    process_variance):
        self.x = np.array([[initial_pos[0]],
                           [initial_pos[1]],
                           [initial_velocity[0]],
                           [initial_velocity[1]]])
        self.x_upper = x_bounds[1]
        self.x_lower = x_bounds[0]
        self.y_upper = y_bounds[1]
        self.y_lower = y_bounds[0]
        self.F = F
        self.Q = process_variance

    #If the ball reaches a boundary in one plane its velocity is reversed in that plane
    def move(self):
        test = self.F @ self.x
        if test[0] >= self.x_upper or test[0] <= self.x_lower:
            self.x[2] *= -1
        if test[1] >= self.y_upper or test[1] <= self.y_lower:
            self.x[3] *= -1
        next = self.F @ self.x

        self.x = next + np.array([[0],[0], [np.random.normal(0, np.sqrt(self.Q))],
                [np.random.normal(0, np.sqrt(self.Q))]])

#Initial Conditions
h = 0.05
max_steps = 2000
processvariance = 0.000001
measurevariance = 0.001
```

```
F = np.array([[1, 0, h, 0],
              [0, 1, 0, h],
              [0, 0, 1, 0],
              [0, 0, 0, 1]])

obj = Ball([0.1,0.05], [0,0], [-1,1], [-1,1], F, processvariance)
sensor = Sensor(measurevariance)

filter = KalmanFilter(4, 2, dim_u=0)
filter.F = F
filter.H = np.array([[1,0,0,0],         #Picks out only x position and
                     [0,1,0,0]])        #y position of the state vector

filter.R = np.eye(2) * measurevariance

#Covariance matrix for process variance, with 0 covariance between x and y directions
filter.Q = np.array([[1, 0, 1, 0],
                     [0, 1, 0, 1],
                     [1, 0, 2, 0],
                     [0, 1, 0, 2]]) * processvariance

filter.x = np.zeros([4,1])                  #Initial position and
                                            #velocity are given as all 0 to the filter
filter.P = np.ones((4,4)) * 500             #Very high initial covariance tells the filter
                                            #the initial position and velocity are
                                            #incorrect and should not be regarded

#Data storage for plots
x_data, y_data = [], []
x_sensor, y_sensor = [], []
x_filter, y_filter = [], []
x_residuals, y_residuals = [], []
K_data_x, K_data_y = [], []
K_data_vx, K_data_vy = [], []

#Simulation process
for i in range(max_steps):
    filter.predict()                                        #Filter produces an a
                                                            #priori estimate
    sense = sensor.sense(np.array([obj.x[0], obj.x[1]]))    #Sensor measures position
                                                            #with some error
    filter.update(sense)                                    #Filter uses measurement to
                                                        #produce a posteriori measurement

    #Storing data
    x_sensor.append(sense[0])
```

```
    y_sensor.append(sense[1])
    x_data.append(obj.x[0])
    y_data.append(obj.x[1])
    x_filter.append(filter.x[0])
    y_filter.append(filter.x[1])
    x_residuals.append(abs(filter.x[0] - obj.x[0]))
    y_residuals.append(abs(filter.x[1] - obj.x[1]))
    K_data_x.append(filter.K[0,0])
    K_data_y.append(filter.K[1,1])
    K_data_vx.append(filter.K[2,0])
    K_data_vy.append(filter.K[3,1])

    obj.move()                                  #Ball moves with some process variance

#Raw Data Plot
plt.plot(x_data, y_data, lw = 2, label="True Position")
plt.plot(x_sensor, y_sensor, color = 'r', ls = '--', lw = 0.8,
label = "Sensor Position Measurements")
plt.plot(x_filter, y_filter, lw = 2, color = 'g', label = "Filtered Position Estimate")
plt.plot([0], [0], marker='*', markersize = 12,  label='Starting Position', color="y")
plt.xlim(-1.1,1.1)
plt.ylim(-1.1,1.1)
plt.hlines(y=1, xmin=-1, xmax=1, colors='b', label="Boundary")
plt.hlines(y=-1, xmin=-1, xmax=1, colors='b')
plt.vlines(x=-1, ymin=-1, ymax=1, colors='b')
plt.vlines(x=1, ymin=-1, ymax=1, colors='b')
plt.title("Raw Data")
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left')
plt.show()

#Residuals Plot
plt.plot(x_residuals, label = 'x resiudals')
plt.plot(y_residuals, label='y residuals')
plt.title("Residuals")
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left')
plt.show()

#Kalman Gain Plot
plt.plot(K_data_x, label = 'x position')
plt.plot(K_data_y, label = 'y position')
plt.plot(K_data_vx, label = 'x velocity')
plt.plot(K_data_vy, label = 'y velocity')
plt.legend(bbox_to_anchor=(1.05, 1.0), loc='upper left')
plt.title("Kalman Gain Data")
plt.ylim(0, 0.1)
```